Programmable In-Network Timing Obfuscation for DNP3

Philip Akekudaga
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A.
philip.akekudaga@uri.edu

Hui Lin
Dept. of Electrical and Computer Engineering
University of Rhode Island
Kingston, RI, U.S.A.
huilin@uri.edu

***Abstract*—Device fingerprinting is an essential step in cyber reconnaissance against industrial control systems (ICSs). In a power grid, a passive observer on the control network can tell the model of a DNP3 outstation and the type of operation it performs just from the timing of its responses, and this timing stays visible even when the payload is encrypted. Because the traffic obfuscation defenses built for general web traffic pad, split, and delay packets on the end host, they do not directly transfer to ICS traffic, since a protocol like DNP3 validates every block of its payload and outstation devices such as protective relays cannot be modified to shape their own traffic. In this paper, we present an in-network timing obfuscation framework that runs in the data plane of a single programmable switch at the outstation edge. The framework classifies each DNP3 transaction, holds the packets that carry the outstation's timing, and releases them at configurable offsets, so that the timing the observer records is a policy value rather than the device's own. We use the two timing features introduced by Formby et al. as case studies: the cross-layer response time (CLRT) of READ and of the SELECT phase of select-before-operate control, and the master-visible timing of an OPERATE. We implement the framework as a P4 program on an Intel Tofino-1 and evaluate it against a physical SEL-751A relay. With the timing mechanism off, the relay's CLRT spreads over roughly 1 to 18 ms and differs between READ and SELECT; with it on, both settle at a median of 4.001 ms with a standard deviation of 0.02 ms. The master-visible OPERATE ACK-to-echo interval stays at about 4.00 ms across three configured internal holds. Consequently, the mutual information between the transaction class and the CLRT falls from 0.42 bits to below 0.003 bits, inside its permutation null, and a READ-versus-SELECT classifier falls from 0.59 balanced accuracy to chance. These results show the suppression of a timing channel on one relay in one capture session; they do not hide the function codes that remain visible, and they do not show indistinguishability across devices.**

***Index Terms*—traffic obfuscation, device fingerprinting, DNP3, programmable switches.**

I. Introduction

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies present network traffic obfuscation [3], [4]. Because device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5], traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods do not directly transfer device fingerprinting in ICS/SCADA environments for two main reasons. First, the leaked features are different. General web traffic obfuscation usually targets the flow level sizes and timing patterns [7] whereas ICS device fingerprinting is carried by the response timing of the protocol exchanges, which stays visible even when the payload is encrypted [10]. Secondly, ICS protocols leave no room or flexibility to add bytes or dummy or fabricated packets. This is because a protocol like DNP3 validates its payloads, so padding and injected packets will break the exchange when the master reassembles the response packets and checks the cyclic redundancy check (CRC) for every sixteen application bytes [11]. Consequently, a defense in this setting must hide the timing that leaks even after encryption while changing no bytes.

In this paper, we present an in-network mechanism that normalizes an outstation's fingerprintable features using the data plane of the Tofino programmable switch at the outstation edge, without changing the endpoints or modifying the bytes of the DNP3 traffic. This defense holds per transaction responses and releases them at configurable offsets so the master-visible timing then becomes a policy value. We implement this as a P4 program on a single switch and evaluate it on

a cyberphysical testbed and make the following contributions:

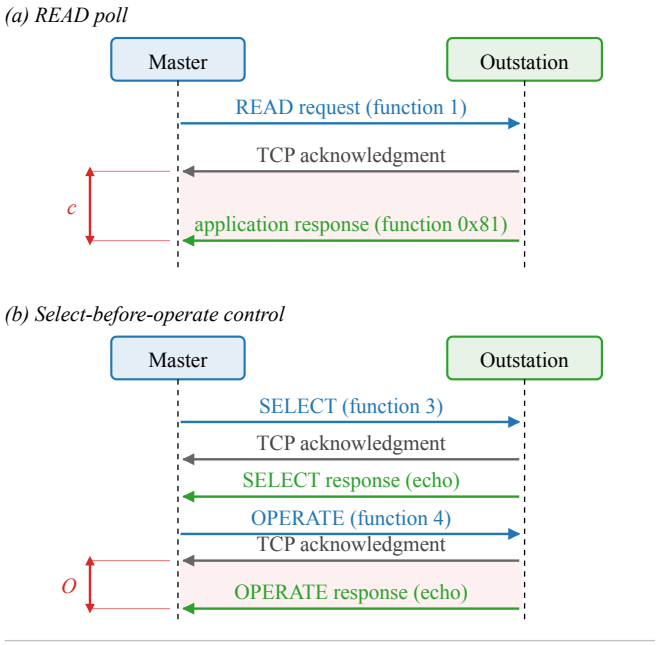
- We propose an in-network turning framework that obfuscates timing features and shapes how an on-path passive observer fingerprints outstation devices. This, to the best of our knowledge, is the first in-network defense for ICS device-fingerprinting features.
- A case study of the timing frameworks introduced by Formby et al [10], we present a multi-configurable mode of operation that holds TCP acknowledgments and DNP3 Responses and releases at predetermined offsets from the request, such that the master visible response time is deterministic
- Normalizes the control command and operation transaction time. As a second case study covering the physical fingerprinting feature of Formby et al. We build a hold mechanism to hold and determine the observed time on the master-facing observed traffic and show the passive observer records a time that is obfuscated
- We evaluate this mechanism on a single physical Tofino and Outstation device.

II. Background and Motivation

In this section we focus on the DNP3 transactions the paper covers, the features that leak from them and how an adversary uses them in fingerprinting, and the opportunity that a programmable data plane gives us to change the timing without touching the endpoints.

The DNP3 read and control transactions. DNP3 carries supervisory control and data acquisition (SCADA) traffic over TCP between a master and an outstation [11]. The master polls the outstation with a READ request (function code 1), and the outstation returns the requested measurements in an application-layer response (function code 0x81). A control command, on the other hand, uses select-before-operate (SBO): the master first sends a SELECT (function code 3) that names an output point, and the outstation responds by echoing the request; the master then sends an OPERATE (function code 4), which the outstation confirms with a response that echoes the command. Because DNP3 runs over TCP, every request is also acknowledged at the transport layer before the application answers. As such, each transaction on the wire is a request, a TCP acknowledgment that carries no DNP3 payload, and a DNP3 response, in that order, as Figure 1 shows. We use these terms consistently in the rest of the paper: the acknowledgment is the TCP segment with the ACK flag that the outstation returns first, and the response is the DNP3 application frame that follows it.

Why the transaction timing is a fingerprint. The acknowledgment is produced by the outstation's TCP stack as soon as the request arrives, whereas the response is produced by its application only after the device has done the work the request asks for. Consequently, the interval between the two, the CLRT, reflects the outstation's internal processing rather than the network path, and it differs between device models and between the kinds of work they do. Formby et al. used it to tell device types and models apart in a substation network, and



c: acknowledgment-to-response interval of the READ (CLRT).
O: master-visible OPERATE ACK-to-echo interval.

Fig. 1. The DNP3 transactions the paper measures, as seen on the master-facing link; requests in blue, transport-layer acknowledgments in grey, and application responses in green. (a) A READ poll: the request, the outstation's transport-layer acknowledgment, and its application-layer response. The interval *c* between the acknowledgment and the response is the cross-layer response time (CLRT). (b) A select-before-operate control: SELECT and OPERATE each produce an acknowledgment and a response that echoes the command. The interval *O* between the acknowledgment and the echo of the OPERATE is the master-visible OPERATE ACK-to-echo interval.

used the time a device takes to complete a control operation as a second feature that reveals the physical device behind the command [10]. Our own measurements on a physical SEL-751A relay show the same effect on one device: with no timing defense, the median CLRT of a READ is 1.272 ms and that of a SELECT is 2.107 ms, both with long tails, so the two transaction classes are already partly separable from timing alone (Section VI). The function codes remain visible in the plaintext, so the classes themselves are not a secret; what the timing adds is a channel that identifies the device and its operation without any payload, and that survives encryption.

Measured and inferred events. Everything the paper measures is a packet timestamp on the master-facing link, i.e., the request, the acknowledgment, and the response. The switch-internal release times and any event on the relay-facing link are not observed; where the paper names them, they are inferred from the configuration. Physical breaker motion is never measured.

Programmable data planes. A programmable switch exposes a match-action pipeline, written in P4 [12], that parses headers, keeps per-packet state in registers, and places packets into output queues at line rate. Because such a pipeline can delay and release a packet without a host in the loop, it can hold an outstation's acknowledgment and response and release

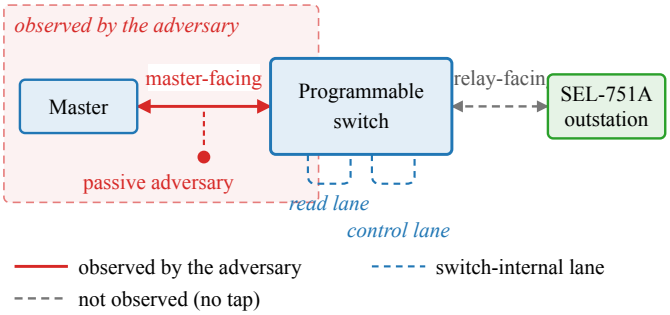


Fig. 2. Observation model. The master reaches the SEL-751A outstation through one programmable switch. The passive adversary sees only the master-facing link (solid red, shaded region); the relay-facing link and the two switch-internal scheduling lanes (dashed) are not observed, by the adversary or by our measurements.

them on a schedule that the operator sets, while changing no endpoint and no byte of the payload.

III. Threat Model and Objectives

Assumptions on the adversary. The adversary is a passive observer on the master-facing link and has the ability to record TCP and IP headers, segment sizes, and arrival times, but neither modifies nor injects packets, as in the fingerprinting attack of Formby et al. [10]. Figure 2 shows the setup. Because DNP3 is usually deployed without transport security across shared substation networks [13], we assume the traffic here is unencrypted. Therefore, the adversary is able to read the exchanges directly, including the DNP3 function codes, and from them it derives the transaction timing of Section II and can train a classifier that separates transaction classes from timing features. The adversary cannot compromise the master or the outstation, control the switch, or read the switch's internal state. It also does not observe the relay-facing link, because in our deployment that link is inside the switch and no tap exists, and it cannot see physical breaker motion. An adversary that actively interacts with the traffic, sends its own probes, or changes the network is out of scope; we stay with the passive observer, which is the setting that prior traffic obfuscation work targets.

Reconnaissance target. The adversary's objective is to fingerprint outstation devices and obtain knowledge to prepare for a later attack. Because control-related attacks such as false data injection attacks work only when the attacker knows the target devices [14], reconnaissance is the enabling step to identify the devices and their weaknesses. The fingerprinting process can succeed without decoding the payloads because it is done using the timing features of the master-outstation exchanges. Following [10], the two timings in scope are the cross-layer response time (CLRT), i.e., the interval between the transport-layer acknowledgment and the application-layer response of a READ or of the SELECT phase of SBO, and the master-visible OPERATE ACK-to-echo interval, i.e., the interval between an OPERATE's acknowledgment and its echo. The CLRT reflects the outstation's internal processing, while the second reflects the physical device behind a control command. The adversary also sees the request-to-acknowledgment interval, which we report as measured.

Research objectives. Our objective is therefore to remove these features from the master-facing observing adversary. We state it as three properties in line with our design and evaluation.

- **RO1 (read timing).** The visible CLRT of READ and of the SELECT phase of SBO is a configurable policy value that is independent of the transaction, so it no longer reflects the outstation's original processing signature.
- **RO2 (control timing).** The visible OPERATE ACK-to-echo interval is a fixed policy value that is independent of the switch's internal release delay applied to the command.
- **RO3 (timing leakage).** The measured timing features carry less information about the transaction class, such that a classifier trained on Timing OFF timing features performs no better than chance on Obfuscated ones.

What is out of scope. The framework does not hide the function codes, which stay visible in plaintext DNP3; RO3 is about the timing channel, not the command semantics. It does not change packet sizes, and no size-related claim is made in this paper. Additionally, it does not claim that two different devices become indistinguishable: the evaluation has one outstation, and what it shows is that this outstation's timing signature is replaced by a policy signature.

IV. Design

In Figure 3, we show the design of the hold and timing obfuscation mechanism. When a request from the master enters the switch, the ingress classifies it by its function code, records the state it needs to recognize the outstation's answer, and forwards the request through the pipeline without a delay. When the outstation answers, the answer goes through the same pipeline, but this time the program holds the acknowledgment and the response in switch queues in the traffic manager and releases each one at a predetermined offset. In this way, the master sees a timing value that the policy sets instead of the timing that the outstation itself produces, which is the signature used for fingerprinting. Because the read and the control responses must not interfere with each other, we implement a separate hold lane for each, and because the two transactions expose different information, each lane uses its own anchor for the offsets. The rest of the section presents the mechanisms and what they require.

Classification and transaction state. The switch parses the DNP3 link and application headers of every packet on the protected session and classifies each packet as a request (READ, SELECT, or OPERATE, by function code), an acknowledgment (the outstation's first segment with the TCP ACK flag after a request), a response (a DNP3 application frame from the outstation), or other. A small set of registers holds the state of the one transaction in flight on the session, i.e., an active-transaction tag, the expected acknowledgment number of the outstation's reply, the armed deadlines, and,

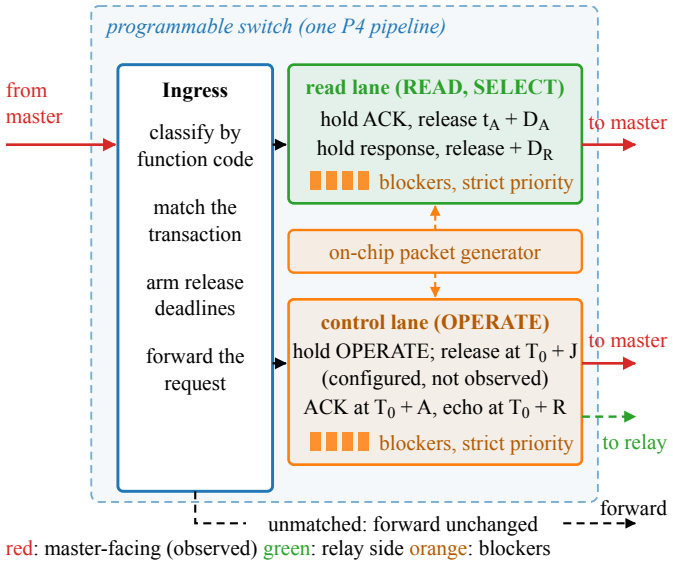


Fig. 3. Design of the hold and timing obfuscation mechanism. Ingress classifies each DNP3 transaction and arms its release deadlines. The read lane holds the acknowledgment and the response of a READ or SELECT and releases them relative to the outstation's acknowledgment. The control lane holds an OPERATE, releases it toward the relay at a configured delay that is not observed, and releases its acknowledgment and echo relative to the request. The generator fills the blocker reservoirs (orange); a strict-priority scheduler drains them first, which realizes every deadline. Red arrows are master-facing and observed; the green dashed arrow is the configured relay-facing release at $T_0 + J$, not observed. Unmatched traffic is forwarded unchanged.

for control, the identity of the held OPERATE. The state is written once per event and read by the packets that follow, so a duplicate acknowledgment or a retransmitted request cannot arm a second deadline.

Anchoring the read deadlines to the acknowledgment. A held packet must leave the switch at a specified deadline, but the switch and P4 have no general-purpose timer. For a READ or a SELECT, the feature to remove is the CLRT, the interval between the acknowledgment and the response. Therefore, when the outstation's acknowledgment reaches the switch at time t_A, the program arms two deadlines from the policy offsets D_A and D_R,

$$t_{\text{ack}} = t_A + D_A, \qquad t_{\text{resp}} = t_A + D_A + D_R, \tag{1}$$

at which it releases the TCP acknowledgment and the DNP3 response to the master respectively. Consequently, the CLRT interval the adversary measures is

$$\text{CLRT} = t_{\text{resp}} - t_{\text{ack}} = D_R, \tag{2}$$

a policy value that replaces the CLRT interval that the outstation's internal processing produces and that serves as its fingerprint. Because the switch can delay a packet but cannot release it before it exists, equation (2) holds only when the response has arrived by $t_A + D_A + D_R$; as such, $D_A + D_R$ must be larger than the largest CLRT the outstation produces on its own for the traffic to be defended, and every response in our captures met that deadline. Note that the request-to-acknowledgment interval then becomes the outstation's own acknowledgment latency plus D_A: it is shifted by a constant, but the sub-millisecond spread of the outstation's TCP stack remains in it.

Anchoring the control deadlines to the request. A control command is different, because the framework also holds the OPERATE itself before it reaches the relay. This separates the moment the master sent the command from the moment the relay acts on it, which an operator may want for a control command; but the outstation's acknowledgment then exists only after the release, so a deadline of the form $t_A + D$, as in (1), would carry the length of the hold into the observable. Therefore, the control path is anchored to the request instead. When the OPERATE arrives at time T_0, the switch holds it and releases it to the relay at an internal delay J, while the acknowledgment and the echo the master sees are placed at fixed offsets from T_0,

$$t_{\text{ack}} = T_0 + A, \qquad t_{\text{echo}} = T_0 + R, \tag{3}$$

where A and R are configured such that A exceeds J plus the outstation's acknowledgment latency, which the control plane checks before it installs the values. The master-visible OPERATE ACK-to-echo interval is therefore

$$O = t_{\text{echo}} - t_{\text{ack}} = R - A, \tag{4}$$

in which J does not appear. J is drawn per transaction from a configured set, so the relay-facing release time varies while the master-facing observable does not. In our experiments $D_R = R - A = 4$ ms, so both case studies present the same 4 ms policy value to the observer.

Security and deadlines. The anchor matters as much as the offsets. On the read path the request is forwarded at once and only the reply is held, so anchoring to the outstation's acknowledgment costs nothing and lets the hold start as late as possible; by (2), the CLRT becomes D_R regardless of how long the outstation took. On the control path the command is held, so the acknowledgment is a function of J, and only a request-anchored rule keeps J out of the observable, by (4). As a result, an adversary that observes and measures either interval learns the policy and not the physical device behind the transaction, and because J is not present in (4), the adversary cannot subtract a known offset to recover the device's original processing time either.

Reservoir tokens as data-plane timers. As the programmable switch has no timer, to achieve a deadline from scheduled packets alone, a held packet must wait in its queue behind a reservoir of blocker packets that sit in a higher-priority queue on the same port. A strict-priority scheduler in the traffic manager drains the blockers first, so the held packet cannot leave until the reservoir is empty. If τ is the time to drain one blocker, a reservoir of K blockers will hold a packet for about $K\tau$, and the control plane sizes K for each deadline. The blockers are generated by the switch's on-chip packet generator when the request arrives, they circulate on internal ports, and they are dropped in the switch when they expire. Consequently, they never leave the switch and are not visible to the master nor the outstation. A tag in each blocker

names the transaction and the lane it belongs to, so a stale blocker from an earlier transaction cannot delay a later one.

Independent scheduling lanes. A read response and a control response can be held at the same time, and if they were serialized in a single lane, whichever holds longer would delay the other and hence reshape the timing policy that is fixed. This therefore gives each treatment its own internal recirculation lane and its own traffic manager queues, which are independent of each other, and the register state of the two lanes is likewise separate.

Unmatched transactions and failing open. A defense on a control path must not become an availability risk. As such, a packet that does not belong to a protected session, a request the framework does not recognize, an acknowledgment with no armed transaction, or a response that arrives after its transaction has retired is forwarded unchanged. In this way, an unexpected exchange loses its protection but not its delivery. Every hold is also bounded: a fail-open budget on the number of blocker passes releases a held packet if its reservoir has not drained within a configured horizon and clears the transaction state, so a lost acknowledgment or response cannot leave a deadline armed forever. With the timing mode switched off, the same program forwards every packet immediately and arms nothing.

Byte preservation. The framework only holds and forwards packets. Since it neither pads a response, injects a packet into the protected exchange, nor edits a DNP3 field, every byte the master reassembles is the byte the relay produced, and the payload's cyclic redundancy checks remain valid.

V. Implementation

Target and program. We implement the design as a single $P4_{16}$ program for the Tofino Native Architecture on an Intel Tofino-1 switch, compiled with the Barefoot SDE 9.13 compiler. The program fits all twelve ingress match-action stages of one pipeline and six egress stages, with 112 tables. The classification of Section IV is a parser for the Ethernet, IP, TCP, and DNP3 link and application headers followed by a chain of match-action tables, and the transaction state is a set of one-entry registers accessed by stateful ALU actions, one access per packet per register. The final forwarding decision is made by one table keyed on a compact outcome computed by the earlier stages, so that every disposition (forward, hold, release, or drop an expired blocker) is an explicit entry.

Ports and lanes. The master is cabled to one front-panel port and the relay to another; these are the only external links. Two further ports are configured in loopback and carry no cable: one is the read lane and the other the control lane of Section IV, each with its own hold and blocker queues at descending strict priority.

Timing state. The deadlines are 32-bit words in units of 256 ns taken from the ingress timestamp of the anchoring packet. On the read path the acknowledgment's arrival is the anchor and the offsets D_A and $D_A + D_R$ are added to it. On the control path the OPERATE's arrival is the anchor and A, R, and J are added to it, and the acknowledgment and echo deadlines are written when the OPERATE is admitted, so that the outstation's later acknowledgment cannot re-anchor them. J is selected per transaction by a table keyed on a data-plane random value from a configured set of admissible holds. The offsets D_A, D_R, A, R, the J set, the fail-open budget, and the timing mode (off or on) are all runtime table entries.

Control plane. A Python control plane over the switch's runtime interface brings up the ports, creates the queues, installs the multicast and generator configuration, writes the timing parameters, and reads every value back. The values are checked before they are installed: the offsets must be multiples of the tick, A must exceed the largest J plus the outstation's acknowledgment latency, and R must exceed A. A failed check leaves the mechanism disabled rather than partly configured. The guarded test harness that issues a physical SELECT and OPERATE permits only two isolated control points on the relay and refuses the breaker-close point.

What ran, exactly. The evidence in Section VI was produced by one program loaded once, whose source and compiled binary are identified by hash in the repository. That program is a combined implementation: besides the timing mechanism described here, it carries a size-shaping datapath that we developed separately and that was enabled in both experimental arms. Because it was on in both arms, it is a held constant of the experiment rather than a difference between them, and the change we report is attributable to the timing mode alone. However, this also means that the arm with the timing mechanism off is not an unmodified relay, and we name the two arms *Timing OFF* and *Obfuscated* for that reason. We do not present a cleaned timing-only source as the program that produced the captures. This paper evaluates timing only; the size datapath is not evaluated, claimed, or figured. A rerun of the same protocol with the size datapath disabled would attribute the effect more sharply and is the first item of future validation.

VI. Evaluation

In this section we evaluate the framework against the three research objectives of Section III, and we end with the limitations of the evidence.

A. Testbed

The testbed consists of one Intel Tofino-1 switch, one physical SEL-751A feeder-protection relay as the DNP3 outstation, and one host that runs the DNP3 master and captures the traffic. The master reaches the relay only through the switch. All captures are taken on the master host, on the master-facing link, with one clock; this is the observation point of the threat model, and it is the only one. The relay-facing port is internal to the switch and has no tap. The relay's TCP stack was configured without the TCP timestamp option so that the observer sees no additional timing field, and no capture carries one.

Conditions. The two arms are *Timing OFF*, in which the loaded program forwards every packet immediately, and *Obfuscated*, in which the same program runs with the timing

TABLE I
CAPTURES AND ANALYSED TRANSACTIONS AFTER THE COLD-START EXCLUSION. ALL CAPTURES ARE MASTER-FACING, FROM ONE SESSION.

arm	class (function code)	requests	analysed
Timing OFF	READ (1)	1000	999
Timing OFF	SELECT phase of SBO (3)	489	488
Obfuscated	READ (1)	600	599
Obfuscated	SELECT phase of SBO (3)	500	499
Obfuscated	OPERATE (4), $J = 2, 6, 12$ ms	3×30	3×30

TABLE II
CLRT PER ARM AND TRANSACTION CLASS, IN MILLISECONDS. STANDARD DEVIATION IS THE POPULATION VALUE; ">12 MS" COUNTS OBSERVATIONS ABOVE 12 MS.

arm	class	n	median	std	max	>12 ms
Timing OFF	READ	999	1.272	1.354	12.275	2
Timing OFF	SELECT	488	2.107	2.530	18.178	11
Obfuscated	READ	599	4.001	0.022	4.098	0
Obfuscated	SELECT	499	4.001	0.021	4.124	0

mechanism on, with $D_A = 20$ ms, $D_R = 4$ ms, $A = 20$ ms, $R = 24$ ms, and $J \in \{2, 6, 12\}$ ms. Both arms ran the same binary with the size-shaping datapath active (Section V); the mode toggle is the only difference between them. All captures come from one session on one day, so the evaluation is transaction-disjoint but not session-disjoint.

B. Data and Extraction

Six master-facing captures support the results (Table I). One Timing OFF capture has 1000 READ and 489 SELECT transactions, two Obfuscated captures have 600 READ and 500 SELECT transactions, and three Obfuscated SBO captures have 30 SELECT and 30 OPERATE transactions each, one per value of J. A transaction is a master request (function 1, 3 or 4), the relay's first segment with the TCP ACK flag after it, and the relay's DNP3 response. Every request in every capture has both, so no transaction is unpaired. The first transaction of each TCP connection is excluded as a cold start, because the relay's first response after a connect is dominated by session setup inside the device; this removes one READ and one SELECT from each capture that opens a connection, and the counts in the table are after that exclusion. Every statistic and figure regenerates from the raw captures with a self-contained script, fixed seeds, and a test suite of 102 checks on pairing, direction, function codes, monotonic timestamps, exclusions and the absence of hard-coded results in the plotting code.

C. Effectiveness in RO1: READ and SELECT CLRT

Why. RO1 asks whether the read-path rule of (1) pins the CLRT of a READ and of a SELECT to the policy value $D_R = 4$ ms on a real relay whose own CLRT varies.

Result. Figure 4 shows the CLRT distributions and Table II the statistics. With the timing mechanism off, the relay's CLRT depends on what it is asked to do: the READ median is 1.272 ms with a standard deviation of 1.354 ms and a maximum of 12.275 ms, whereas the SELECT median is 2.107 ms with a standard deviation of 2.530 ms and a maximum of 18.178 ms, and 2 READ and 11 SELECT observations lie above 12 ms. With the mechanism on, both classes settle at a median of 4.001 ms with standard deviations of 0.022 and 0.021 ms and maxima of 4.098 and 4.124 ms, and no observation lies above 12 ms. The bootstrap 95 % interval on the Obfuscated median is [4.000, 4.001] ms for READ and [4.001, 4.002] ms for SELECT. The reason the two classes coincide is (2): once the response has arrived before its deadline, the observer sees D_R regardless of how long the relay took, and every response in these captures arrived in time. The 1 μs residual above 4.000 ms and the 0.02 ms spread come from the release jitter of the scheduler, not from the relay.

Distributions. Figure 5 shows the same data as empirical distribution functions, with the Timing OFF tail preserved to its largest value of 18.178 ms. The Jensen-Shannon distance between the READ and SELECT distributions, on a predeclared common grid of 0.2 ms bins, falls from 0.676 with the mechanism off to 0.046 with it on. Additionally, the distance between the Timing OFF and Obfuscated distributions of the same class is 0.997 for READ and 0.915 for SELECT, i.e., the Obfuscated distribution shares almost no support with the Timing OFF one.

Timing-feature overlap. Figure 6 places every transaction by the two timing features the observer can compute directly from its timestamps, i.e., the request-to-acknowledgment interval and the CLRT. With the mechanism off, READ and SELECT occupy different regions: the acknowledgment arrives after a median of 0.627 ms for READ and 0.492 ms for SELECT, and the CLRT separates the classes as above. With the mechanism on, the CLRT of both classes is 4.001 ms, and the request-to-acknowledgment interval moves to a median of 20.661 ms for READ and 20.510 ms for SELECT, which is D_A plus the relay's own acknowledgment latency, as (1) predicts. That interval is shifted, not pinned: its standard deviation is 0.457 and 0.616 ms, the spread of the relay's TCP stack, and the inset shows it. The overlap the figure displays is therefore in the CLRT, the feature RO1 targets, together with a shift of the second feature by a constant.

D. Effectiveness in RO2: Master-Visible OPERATE ACK-to-Echo Interval

Why. RO2 asks whether the control-path rule of (3) keeps the master-visible OPERATE ACK-to-echo interval $O = R - A$ independent of the hold J applied to the OPERATE, on the real relay, where the relay's acknowledgment can only exist after the command has been released to it.

Result. Figure 7 shows the master-visible acknowledgment-to-echo interval for 30 OPERATE transactions at each configured hold. The medians are 4.001, 4.002 and 4.003 ms at $J = 2$, 6 and 12 ms, with a standard deviation of about 0.026 ms in each condition, and the bootstrap intervals on the medians overlap. The absolute intervals from the request are about 21.0 ms to the acknowledgment and 25.0 ms to the echo in every condition, i.e., the configured $A = 20$ ms and

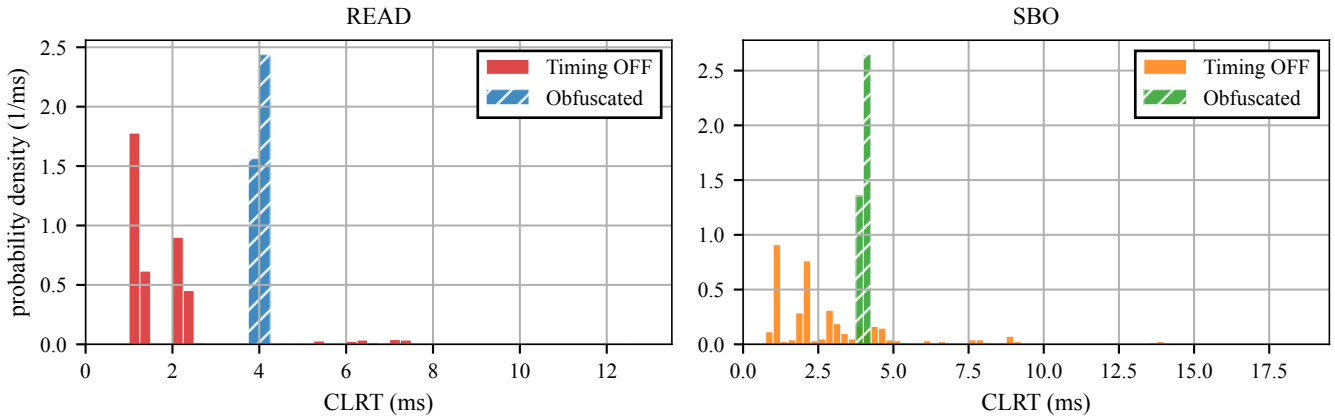


Fig. 4. CLRT, the interval from the outstation's TCP acknowledgment to its DNP3 response, with the timing mechanism off (Timing OFF) and on (Obfuscated), measured master-facing against the physical SEL-751A. Left: READ (function 1). Right: the SELECT phase of select-before-operate (function 3), written SBO in the legend; these are SELECT observations, not complete SBO transactions. Timing OFF: 999 READ and 488 SELECT transactions; Obfuscated: 599 and 499. Each panel's horizontal axis runs to its largest observation, so the full tail is visible. Both arms ran the same switch binary with the size-shaping datapath active, so Timing OFF is not an unmodified device baseline.

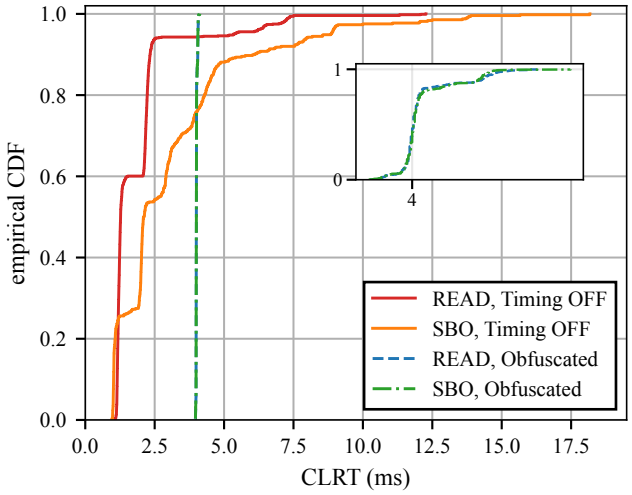


Fig. 5. Empirical CDF of CLRT for READ and for the SELECT phase of SBO, with the timing mechanism off (solid) and on (dashed). With the mechanism off the two classes separate and the SELECT distribution carries a tail to 18.18 ms, shown in full; with it on, both collapse onto 4.001 ms, and the inset magnifies the two Obfuscated curves, which coincide at full scale. Size shaping was active in both arms.

$R = 24$ ms plus a path and capture offset of about 1 ms that cancels in the difference. The reason J does not appear is (4): both deadlines are armed from the OPERATE's arrival, so a sixfold change in the hold moves neither. What this result establishes is what the master sees. J was configured, not measured: the relay-facing release at $T_0 + J$ was never captured, the number of copies that reached the relay was not observed, and the physical breaker was not operated, so this is a master-visible timing result and not a physical operating-time result. The master issued exactly one OPERATE per transaction, and the relay's outputs remained open before, during and after every batch.

E. Effectiveness in RO3: Transaction-Class Timing Leakage

Why. RO3 asks how much information about the transaction class the CLRT carries before and after the defense. The class labels are READ versus the SELECT phase of SBO, the two classes the captures contain. We stress that the function codes remain visible to the observer in the plaintext. The question is whether the timing channel carries the class on its own, which is what matters when the payload is encrypted or when timing is used to confirm what the payload says.

Result. Figure 8 reports two measures. We estimate the mutual information between class and CLRT on a predeclared grid of 60 bins over 0 to 12 ms and compare it with a permutation null of 1000 label shuffles. It is 0.424 bits with the mechanism off, far above its null interval of [0.015, 0.033] bits, and it falls to below 0.003 bits with the mechanism on, inside its null interval of [0.000, 0.003] bits. We quote the Obfuscated estimate as a bound rather than a point value, because the grid places a bin edge at exactly 4.000 ms and 18 % of the Obfuscated observations lie within a microsecond of it, so the point estimate moves between 0.000 and 0.002 bits with the grid phase; under every phase tested, it stays inside the null. We also fit a logistic regression on the single feature CLRT to a 60 % transaction-disjoint split of the Timing OFF data. It reaches a balanced accuracy of 0.592 on the held-out Timing OFF transactions, with a bootstrap 95 % interval of [0.558, 0.626], and, applied without refitting to the Obfuscated data, it falls to 0.500, the chance baseline, with a degenerate interval because it predicts one class for every transaction. This is because the two Obfuscated distributions coincide within the classifier's resolution. We interpret this as the suppression of a timing channel: on this relay the CLRT no longer tells READ from SELECT. It is not the concealment

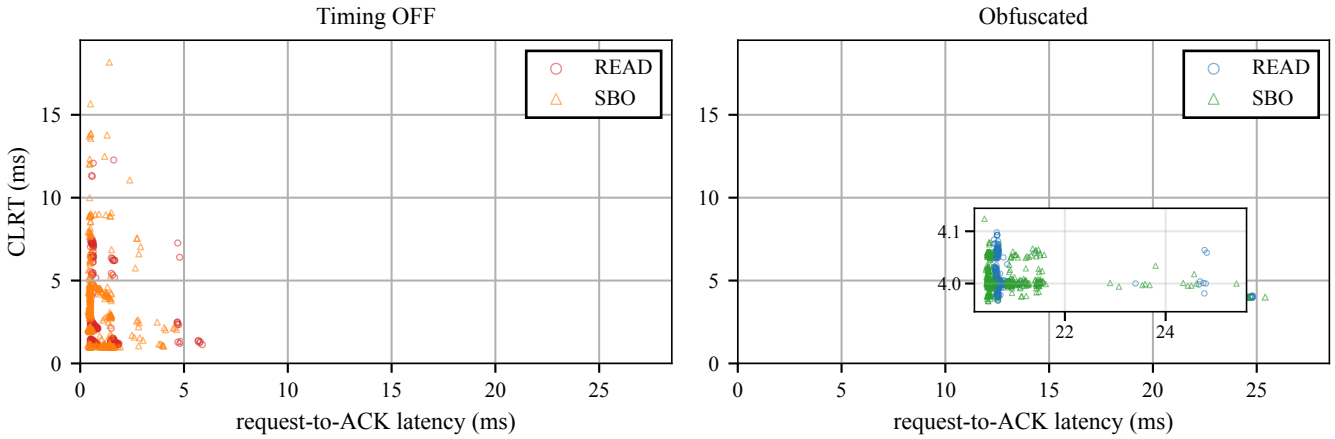


Fig. 6. Timing-feature overlap with the timing mechanism off and on. Each point is one transaction, placed by the two latencies a passive observer measures master-facing: request-to-acknowledgment on the horizontal axis and CLRT on the vertical. Left, Timing OFF: READ and the SELECT phase of SBO occupy visibly different regions. Right, Obfuscated: both classes fall in one small region, magnified in the inset. This is a display of feature overlap between two transaction classes on one relay, not a clustering result and not device identification; no unsupervised algorithm is fitted.

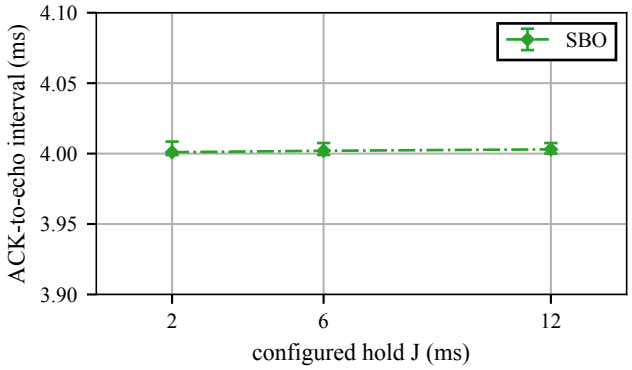


Fig. 7. Master-visible OPERATE ACK-to-echo interval, the interval between an OPERATE's acknowledgment and its echo, under three configured holds. Markers are medians of 30 OPERATE transactions per condition; bars are bootstrap 95 % confidence intervals on the median and are smaller than the markers. The interval stays at the 4 ms policy value across $J = 2$, 6 and 12 ms. J is the configured codebook value; it was not observed on the relay-facing wire, and exactly-once relay delivery is not demonstrated.

of the command semantics, which the plaintext still exposes, and not an identification result across devices.

F. Limitations

The evidence bounds our claims in the following ways, and we state them here rather than in a footnote. First, the testbed has one SEL-751A, one Tofino-1, and one master-facing capture session, so the evaluation is transaction-disjoint rather than session-disjoint, and the results show the replacement of this relay's timing signature by a policy signature, not indistinguishability across devices. Second, the DNP3 function codes remain visible to the observer; the framework suppresses the timing channel and does not hide what the exchange is. Third, size shaping was active in both arms of every capture, so no pure size-free native baseline exists in the data; the mode toggle is the only difference between the arms, but the Timing OFF numbers are the relay's CLRT through that datapath. A rerun with the size datapath disabled is the first item of future validation. Fourth, the configuration provenance is partial: one archived configuration readback reports a failed assertion while every row it shows passes, and the failing assertion cannot be recovered from the archived material. The parameters the timing claims rest on appear as passing rows that match a configure-all run that passed, the read-path offsets are the control plane's defaults corroborated by the wire, and the hold set and the shaping state are established from the captures themselves. Fifth, the relay-facing release at $T_0 + J$ was not captured, physical breaker motion was not measured, and exactly-once delivery of the OPERATE to the relay was not established. Sixth, the request-to-acknowledgment interval of a READ or SELECT is shifted by a constant but still carries the relay's sub-millisecond acknowledgment jitter (Section VI-C). Last, generalization to other outstations and to multiple devices was not evaluated.

VII. Related Work

Device fingerprinting. Devices can be identified from what they emit, even without any access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock as seen in TCP timestamps [15], and GTID fingerprints a device and its type from the timing signature of its packets [16]. These works define the threat we address; our objective, on the other hand, is to remove the timing features they rely on from the master-facing observation.

ICS and power-system fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time, measured at the device, of DNP3 outstations [10], and Gu et al. added device physics as a feature [17]. Jeon et al. fingerprint SCADA

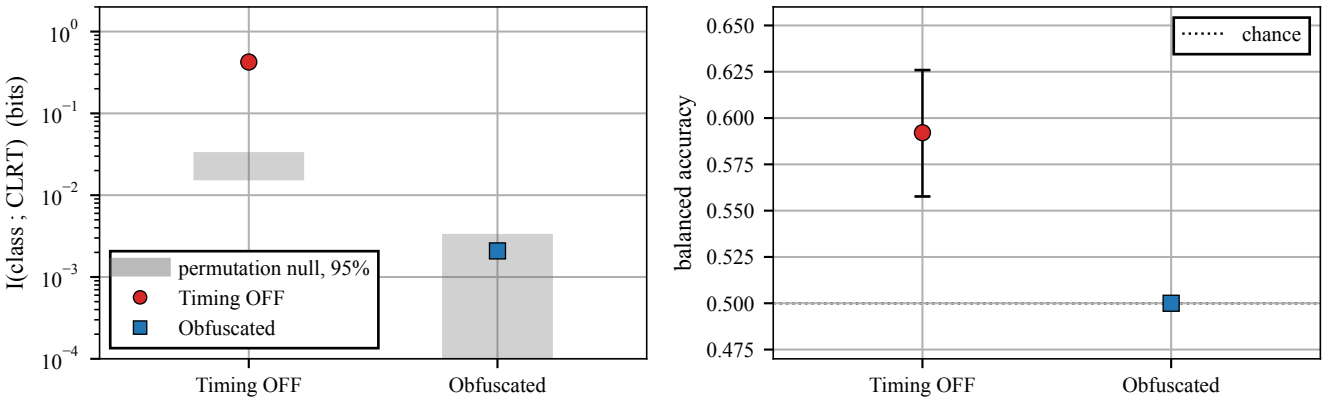


Fig. 8. Timing-only leakage with the timing mechanism off and on. Left: mutual information between transaction class (READ versus the SELECT phase of SBO) and CLRT, with the shaded band giving the 95 % permutation null for each arm. Right: balanced accuracy of a classifier trained on Timing OFF CLRT and applied unchanged to Obfuscated traffic, with the 0.5 chance baseline. Both panels describe transaction-class feature suppression on one SEL-751A in one capture session with a transaction-disjoint, not session-disjoint, split; neither is a device-identification result.

devices passively without deep packet inspection [18]. The reconnaissance these techniques serve enables false-data injection and process-control attacks [14], [19]. Defenses in this setting have so far worked at the content and connectivity layer: DefRec virtualizes physical functions to present deceptive content [20], RAINCOAT randomizes data acquisition and injects decoy measurements [21], and a companion testbed evaluates such anti-reconnaissance approaches on grid infrastructure [22]. Compared to these, our framework works below the semantics they reshape and complements them: it changes when the wire carries the outstation's answer, not what the answer says.

General traffic obfuscation. Traffic morphing transforms one class's packet-size distribution into another's [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [6], and WTF-PAD pads inter-packet gaps adaptively [23]. However, deep fingerprinting attacks defeat several of these [5], and their accuracy at Internet scale is contested [24]. The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [25], and Pacer and NetShaper shape a tenant's traffic to a schedule that bounds side-channel leakage in the cloud [26], [27]. These defenses assume an encrypted payload and a cooperating end host or hypervisor that can add cover traffic. Our framework, in contrast, targets a plaintext protocol whose feature is one interval inside a request-response exchange, adds no byte, and runs where the outstation cannot be changed.

Programmable data-plane traffic transformation. P4 defines the programmable parser and match-action model we use [12]. PIFO defines programmable scheduling at line rate [28], and SP-PIFO approximates it with strict-priority queues on today's switches [29], which is the primitive our reservoirs use. Ditto pads and injects chaff to a fixed pattern at line rate [7], Minos morphs and schedules encrypted traffic on a programmable switch with low overhead [8], and Securitas mixes fragmentation and insertion under a learned policy across several data planes [9]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [30]. Closest to our setting, Hu et al. use programmable switches to enhance industrial-protocol security [31]. Our framework differs from these in the observable it addresses, i.e., the master-visible timing of a DNP3 transaction, and in holding packets rather than adding or reshaping them.

DNP3 timing and security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [13], and specification-based and state-based intrusion detection has been adapted to DNP3 [32], [33]. These do not change the timing an observer sees. The physical operation-time feature we address on the control path is the one Formby et al. measured on a physical outstation [10]; we report what the master sees under the defense and, unlike a measurement of the device, we do not observe the relay-facing side.

VIII. Conclusion

A passive observer of legacy DNP3 traffic can fingerprint an outstation from the timing of its responses, and the size- and delay-based obfuscation built for encrypted web traffic does not transfer to a plaintext control protocol on devices that cannot be changed. In this paper, we presented an in-network timing obfuscation framework that runs in the data plane of a single programmable switch. It holds the packets that carry an outstation's timing and releases them at configured offsets, with one rule for polls and the SELECT phase of control that is anchored to the outstation's acknowledgment, and another rule for OPERATE that is anchored to the request. We implemented it as one P4 program on a Tofino-1 and evaluated it against a physical SEL-751A relay. The framework pinned the CLRT of READ and SELECT to 4.001 ms with a 0.02 ms standard deviation, where the relay's own CLRT spread over 1 to 18 ms; it kept the master-visible OPERATE ACK-to-echo interval at

4.00 ms across configured holds of 2, 6, and 12 ms; and it reduced the mutual information between the transaction class and the CLRT from 0.42 bits to inside its permutation null, with a READ-versus-SELECT classifier falling from 0.59 balanced accuracy to chance. These results show the suppression of a timing channel on one relay in one session; the function codes stay visible, the relay-facing side was not observed, and size shaping was active in both arms of the frozen evidence. As the next validation, we plan to rerun the same protocol with the size datapath disabled and a relay-facing tap, so that the timing mechanism is isolated outright and the release at $T_0 + J$ is measured rather than configured.

REFERENCES

[1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the Cyber Attack on the Ukrainian Power Grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016.

[2] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail," in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012.

[5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.

[6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.

[7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN Traffic Obfuscation at Line Rate," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.

[8] Z. Wang, Q. Li, G. Xie, D. Zhao, K. Li, Z. Fan, L. Ma, and Y. Jiang, "Minos: A Lightweight and Dynamic Defense against Traffic Analysis in Programmable Data Planes," in *Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*, 2025.

[9] G. Xie, Q. Li, Z. Shi, G. Antichi, Y. Zhu, K. Li, C. Weng, S. Miano, Y. Jiang, and M. Xu, "Defending against Traffic Analysis Attacks with Flexible In-Network Obfuscation," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, pp. 2043–2063.

[10] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016.

[11] IEEE, "IEEE Standard for Electric Power Systems Communications—Distributed Network Protocol (DNP3)," 2012.

[12] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[13] S. East, J. Butts, M. Papa, and S. Shenoi, "A Taxonomy of Attacks on the DNP3 Protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol.\ 311)*. Springer, 2009, pp. 67–81.

[14] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks against State Estimation in Electric Power Grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.

[15] T. Kohno, A. Broido, and K. C. Claffy, "Remote Physical Device Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.

[16] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A Technique for Physical Device and Device Type Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.

[17] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for Cyber-Physical System Security: Device Physics Matters Too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, Sep. 2018.

[18] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive Fingerprinting of SCADA in Critical Infrastructure Network without Deep Packet Inspection," arXiv:1608.07679 [cs.CR], Aug. 2016.

[19] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks Against Process Control Systems: Risk Assessment, Detection, and Response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.

[20] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing Physical Function Virtualization to Disrupt Reconnaissance of Power Grids' Cyber-Physical Infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.

[21] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of Network Communication in Power Grid Cyber Infrastructure to Mislead Attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.

[22] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-Physical Testbed: Case Study to Evaluate Anti-Reconnaissance Approaches on Power Grids' Cyber-Physical Infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.

[23] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an Efficient Website Fingerprinting Defense," in *Computer Security – ESORICS 2016*, 2016.

[24] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website Fingerprinting at Internet Scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[25] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the Smart Home Private with Smart(er) IoT Traffic Shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019.

[26] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud," in *Proceedings of the 31st USENIX Security Symposium*, 2022.

[27] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A Differentially Private Network Side-Channel Mitigation System," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[28] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, Agrawal, Anurag, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable Packet Scheduling at Line Rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.

[29] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating Push-In First-Out Behaviors using Strict-Priority Queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.

[30] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and Practical Network Topology Obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.

[31] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial Network Protocol Security Enhancement Using Programmable Switches," in *Proceedings of the IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.

[32] H. Lin, A. Slagell, C. D. Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a Specification-Based Intrusion Detection System for the DNP3 Protocol," in *Proceedings of the Eighth Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.

[33] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 State-Based Intrusion Detection System," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.